# Day 1C — Training Pipeline & Inference

> 🎯 **Goal**: Understand how LLMs are trained (pre-training → SFT → RLHF) and how they generate text at inference time.

> ⏱️ **Time**: 1h theory + 1h practice

> 📦 **Deliverable**: A sampling strategy visualizer + tokenization explorer + clear mental model of the 3-phase training pipeline

---

## Table of Contents


1. Positional Encoding
2. The Training Pipeline
3. Inference — How Generation Actually Works
4. Practice Exercises
5. Deliverable Checklist


---

## Positional Encoding

### Why Position Matters

Transformers process all tokens in **PARALLEL** — every token attends to every other token simultaneously. This is what makes them fast (unlike RNNs, which go left-to-right, one step at a time). But it creates a critical problem:

```
Without positional info, the model sees:

  "dog bites man"  ↔  "man bites dog"

as the SAME input! The set of tokens is identical.
The model has NO concept of order. None. Zero.

It's like dumping a bag of Scrabble tiles on the table —
you know WHICH letters exist, but not what order they go in.
```

We need to **inject** position information into the embeddings.

---

## Sinusoidal Encoding (Original Transformer Paper)

**The clock analogy**: Think of telling time. The seconds hand spins fast, the minutes hand slower, the hours hand slowest. Each "frequency" captures position at a different granularity. Sinusoidal encoding does the same thing — each dimension of the embedding oscillates at a different frequency.

```
Formula:

  PE(pos, 2i)     = sin(pos / 10000^(2i/d))
  PE(pos, 2i+1)   = cos(pos / 10000^(2i/d))

Where:
  pos = position in the sequence (0, 1, 2, ...)
  i   = dimension index
  d   = embedding dimension (e.g., 512)
```

**Why sin AND cos?** Because the model can learn to compute **RELATIVE** positions via linear transformation. `PE(pos+k)` can be expressed as a linear function of `PE(pos)` — this means the model can figure out "how far apart are these two tokens?" just from the encodings.

```
Intuition — it's like binary counting, but smooth:

  Position 0:  [0.00, 1.00, 0.00, 1.00, 0.00, 1.00, ...]
  Position 1:  [0.84, 0.54, 0.02, 1.00, 0.00, 1.00, ...]
  Position 2:  [0.91, -0.4, 0.03, 1.00, 0.00, 1.00, ...]
                 ↑ changes fast       ↑ changes slowly

  Low dimensions  = fast oscillation  (like seconds hand)
  High dimensions = slow oscillation  (like hours hand)
```

---

## Learned Position Embeddings (GPT-2 Style)

Just learn a separate embedding vector for each position. Position 0 gets one vector, position 1 gets another, etc.

```
Simple lookup table:

  position_embeddings = Embedding(max_positions, d_model)

  Position 0 → [0.12, -0.34, 0.56, ...]  ← learned during training
  Position 1 → [0.08, -0.21, 0.43, ...]  ← learned during training
  ...
  Position 2047 → [...]                   ← last supported position
  Position 2048 → ???                     ← CANNOT extrapolate!
```

**Pro**: Simple, works well within training length. **Con**: Can't handle sequences longer than seen in training. Position 2048 is a hard wall.

---

## RoPE (Rotary Position Embeddings)

Used in **LLaMA 3/4**, **Mistral**, **GPT-NeoX**, **Gemini**, and most modern models. This is the current state of the art.

**Key insight**: Instead of ADDING position info to embeddings, **ROTATE** the embedding vectors. Position becomes a rotation angle.

```
Traditional:  embedding = token_embedding + position_embedding
RoPE:         embedding = Rotate(token_embedding, angle=f(position))

Think of it as spinning a vector in 2D space:

  Position 0: →        (no rotation)
  Position 1: ↗        (rotated slightly)
  Position 2: ↑        (rotated more)
  Position 3: ↖        (rotated even more)
```

**Why this is brilliant**: When you compute Q·K (the attention score), the rotation naturally encodes the **DISTANCE** between tokens. Two tokens at positions 5 and 7 will have a rotation difference of 2, regardless of their absolute positions. Closer tokens → smaller rotation difference → naturally captures locality.

**Advantage over alternatives**: RoPE naturally captures relative position AND can extrapolate to longer sequences than seen in training (with techniques like NTK-aware scaling or YaRN).

---

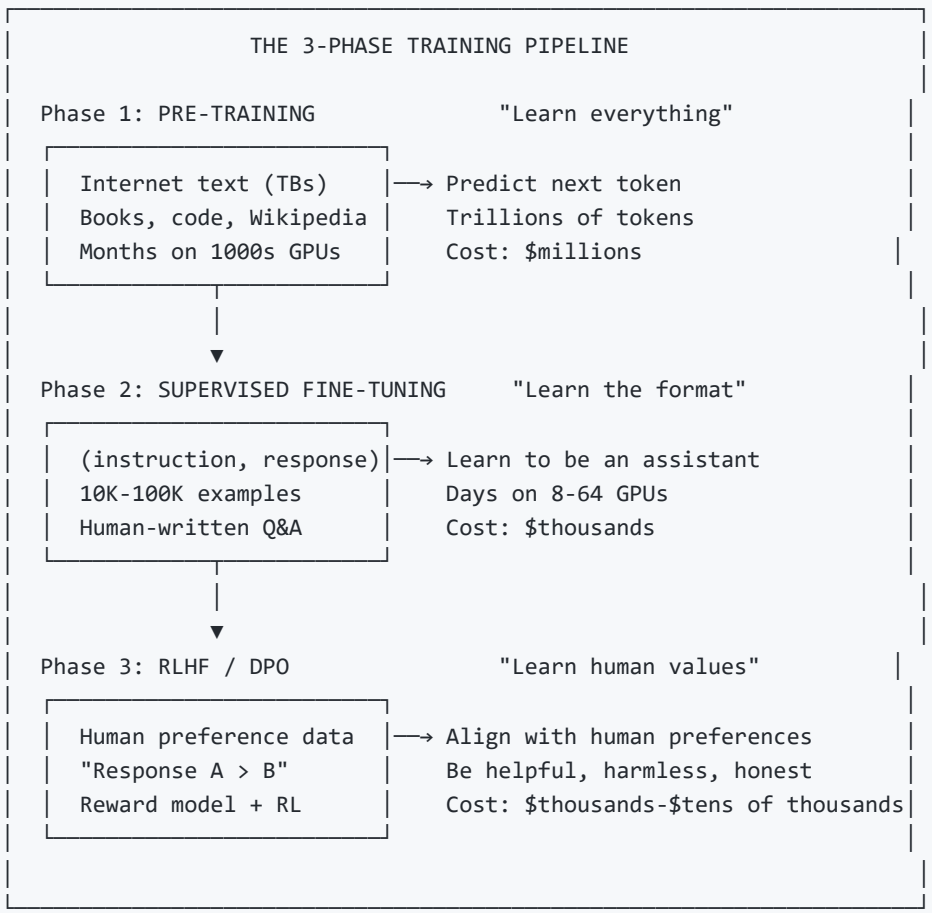
## Comparison: Positional Encoding Methods

```
┌────────────────────┬─────────────────┬──────────────────┬────────────────────┐
│                    │ Sinusoidal      │ Learned          │ RoPE               │
├────────────────────┼─────────────────┼──────────────────┼────────────────────┤
│ Extrapolation      │ Theoretically   │ Cannot           │ Yes (with          │
│ beyond training    │ yes, but weak   │ extrapolate      │ scaling tricks)    │
│                    │                 │                  │                    │
│ Relative position  │ Via linear      │ No (absolute     │ Built-in           │
│ awareness          │ transform       │ only)            │ (Q·K encodes       │
│                    │                 │                  │ distance)          │
│                    │                 │                  │                    │
│ Trainable?         │ No (fixed)      │ Yes              │ No (but can        │
│                    │                 │                  │ tune frequency)    │
│                    │                 │                  │                    │
│ Modern usage       │ Original        │ GPT-2, BERT      │ LLaMA 4, GPT-5,    │
```

```
│                   │ Transformer   │                │ Claude, Gemini  │
└───────────────────┴───────────────┴────────────────┴─────────────────┘
```

## The Training Pipeline

Modern LLMs go through **three distinct phases**, each serving a fundamentally different purpose. Think of it as: learn language → learn format → learn values.




### Phase 1: Pre-training — Learning Language

The model's job is deceptively simple: **predict the next token**. Over and over, trillions of times.

```
Training example:

  Input:  "The capital of France is"
  Target: "Paris"
```

```
The model sees TRILLIONS of such examples scraped from the internet:
books, Wikipedia, code, forums, news — everything.
```

### Cross-Entropy Loss — Intuition

The model outputs a probability distribution over ALL tokens in the vocabulary (e.g., 100,000 probabilities). The correct next token should have HIGH probability.

**Cross-entropy measures: "How surprised is the model by the correct answer?"**

```
Example: "The capital of France is ___"

  Model's prediction (probability distribution over 100K tokens):

  Token         Probability     -log(p)
  _____         ___________     ________
  "Paris"         0.95          0.05   ← low loss (good! not surprised)
  "the"           0.02          3.91
  "Lyon"          0.01          4.61
  "Berlin"        0.005         5.30
  ... (99,996 other tokens share the remaining 0.015)

  Loss = -log(P("Paris")) = -log(0.95) = 0.05  ← Great!

  ________________________________________________

  Bad model's prediction:

  "Paris"         0.01          4.61   ← high loss (bad! very surprised)
  "the"           0.30          1.20
  "banana"        0.15          1.90

  Loss = -log(P("Paris")) = -log(0.01) = 4.61  ← Terrible!
```

**Key insight**: The log function penalizes **confident wrong answers** far more than uncertain ones. If the model says P("Paris") = 0.001 with high confidence in something else, the loss EXPLODES. This forces the model to be well-calibrated.

### What Emerges at Different Scales

```
┌──────────────────────┬──────────────────────────────────────────┐
│ Scale                │ Capabilities that emerge                 │
├──────────────────────┼──────────────────────────────────────────┤
│ Small (1B params)    │ Grammar, basic facts, simple completion  │
│ Medium (7–13B)       │ Reasoning, coding basics, following intent │
│ Large (70–405B)      │ Complex reasoning, few-shot learning,    │
│                      │ nuanced understanding                    │
│ Frontier (MoE /      │ Emergent abilities: chain-of-thought,    │
│  undisclosed scale)  │ in-context learning, self-correction,    │
│                      │ agentic behavior (GPT-5, Claude Opus 4.6) │
```

```
└──────────────────────┴──────────────────────────────────────────────┘

Training data scale:
  GPT-3:      300 billion tokens
  LLaMA-2:    2 trillion tokens
  LLaMA-3:    15 trillion tokens
  LLaMA-4:    ~30+ trillion tokens (estimated)
  GPT-5.x:    undisclosed, likely 20T+ tokens
  Modern:     10T–50T+ tokens
```

**Cost**: Millions of dollars, thousands of GPUs, weeks to months of continuous training. Pre-training is the most expensive phase by far.

---

## Phase 2: Supervised Fine-Tuning (SFT) — Learning the Format

**The problem**: After pre-training, the model is a **COMPLETOR**, not an **ASSISTANT**. It has absorbed immense knowledge, but it doesn't know it should *answer questions*.

```
What pre-training produces:

  User:  "What is 2+2?"
  Model: "This is a common math question often asked in elementary
          school. Teachers typically introduce addition in the first
          grade, and students learn that..."
          ↑ Continues the text, doesn't ANSWER

What we want:

  User:  "What is 2+2?"
  Model: "4"
          ↑ Direct answer
```

**Solution**: Fine-tune on curated (instruction, response) pairs.

```
SFT training data looks like:

  {"instruction": "What is the capital of Japan?",
   "response": "The capital of Japan is Tokyo."}

  {"instruction": "Write a Python function to reverse a string.",
   "response": "def reverse_string(s):\n    return s[::-1]"}

  {"instruction": "Summarize this article: ...",
   "response": "The article discusses..."}

  Dataset size: 10K-100K examples (MUCH smaller than pre-training)
```

**Critical insight**: The model's knowledge doesn't really change during SFT — it already KNEW the answers from pre-training. SFT teaches it the **FORMAT** of being helpful: start with an answer, be concise, follow instructions, use the right tone. It's like a brilliant expert who finally learns presentation skills.

---

## Phase 3: RLHF / DPO — Learning Human Values

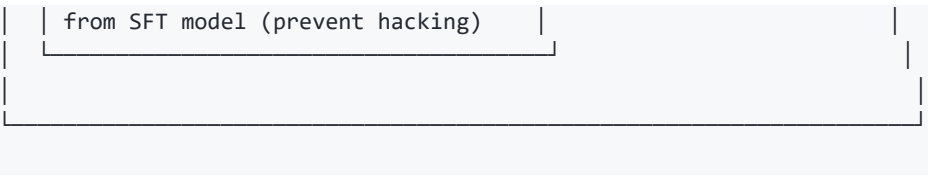
**Why SFT isn't enough**: The model follows instructions now but may still be:

- **Harmful**: generates dangerous or offensive content
- **Unhelpful**: verbose, off-topic, hedging too much
- **Inconsistent**: great at some topics, terrible at others

We need to align the model with **human preferences** — not just "what is correct" but "what do humans actually want?"

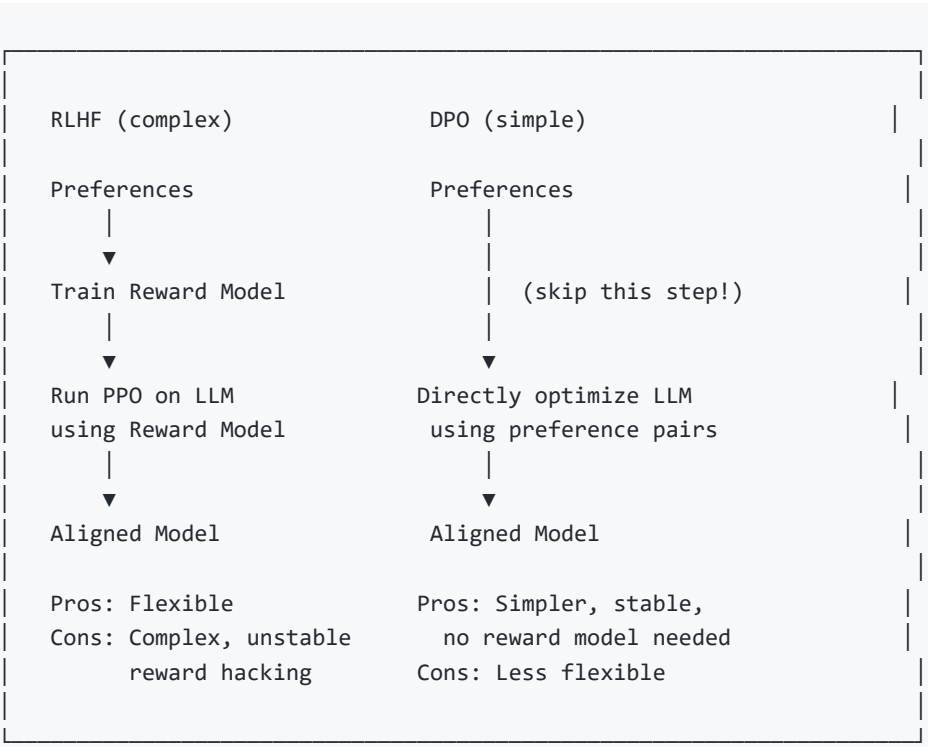
### RLHF (Reinforcement Learning from Human Feedback)






**KL divergence penalty**: Without this, the model learns to "hack" the reward model — generating nonsensical outputs that happen to score high. The KL penalty says: "You can improve, but stay close to the SFT model."

**DPO (Direct Preference Optimization) — The Simpler Alternative**



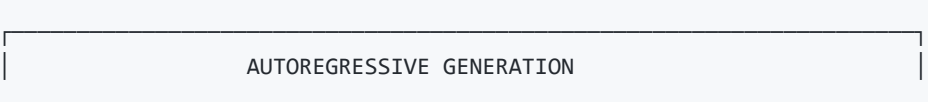

**DPO** skips the reward model entirely. It directly uses the preference pairs (chosen response vs. rejected response) to optimize the LLM. Simpler, more stable, increasingly popular. Used by Meta (LLaMA-3/4), Anthropic (Claude Sonnet/Opus), and OpenAI (GPT-5 series).

---

# Inference — How Generation Actually Works

## Autoregressive Generation

LLMs generate text **one token at a time**. Each generated token is fed back as input for the next step. This is called **autoregressive** generation.





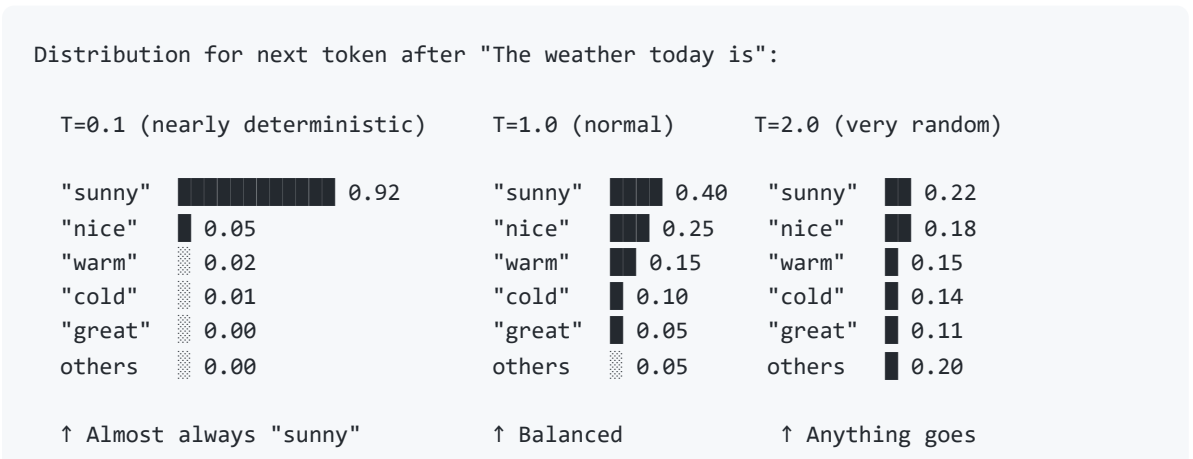

---

## Sampling Strategies

When the model outputs a probability distribution, HOW do we pick the next token? This choice dramatically affects output quality and creativity.

### Temperature

**Temperature controls randomness**. It scales the logits (raw scores) before applying softmax.



```
  Use for: factual Q&A            Use for: general      Use for: creative
                                                         writing, brainstorm
```

- **T → 0**: Always pick the highest probability token (greedy/deterministic)
- **T = 1**: Sample from the actual learned distribution
- **T > 1**: Flatten the distribution, making unlikely tokens more probable

### Top-k Sampling

Only consider the **k highest-probability** tokens, then sample among them. Everything else gets probability 0.

```
Full distribution (10 tokens shown):

  "sunny"   0.40
  "nice"    0.25
  "warm"    0.15    ← Top-3 boundary
  ──────────────
  "cold"    0.10    ← zeroed out
  "great"   0.05    ← zeroed out
  "awful"   0.03    ← zeroed out
  ...

After top-k (k=3), renormalized:

  "sunny"   0.50   (0.40 / 0.80)
  "nice"    0.31   (0.25 / 0.80)
  "warm"    0.19   (0.15 / 0.80)
  others    0.00

k=1  → greedy decoding (always pick the best)
k=50 → moderate diversity (GPT-2 default)
```

### Top-p (Nucleus Sampling)

Pick the **smallest set of tokens** whose cumulative probability ≥ p. The set size ADAPTS to the distribution.

```
Example 1: Confident distribution (p=0.9)

  "Paris"   0.85  ← cumulative: 0.85
  "Lyon"    0.06  ← cumulative: 0.91 ≥ 0.9 → STOP
  "London"  0.04  ← zeroed
  ...
  → Only 2 tokens considered

Example 2: Uncertain distribution (p=0.9)

  "the"     0.15  ← cumulative: 0.15
```

```
  "a"       0.12  ← cumulative: 0.27
  "its"     0.10  ← cumulative: 0.37
  "his"     0.09  ← cumulative: 0.46
  "their"   0.08  ← cumulative: 0.54
  "my"      0.07  ← cumulative: 0.61
  "her"     0.07  ← cumulative: 0.68
  "our"     0.06  ← cumulative: 0.74
  "your"    0.05  ← cumulative: 0.79
  "some"    0.05  ← cumulative: 0.84
  "this"    0.04  ← cumulative: 0.88
  "that"    0.03  ← cumulative: 0.91 ≥ 0.9 → STOP
  → 12 tokens considered!
```

**Why top-p is generally preferred over top-k**: Top-p ADAPTS. When the model is confident, it considers few tokens. When uncertain, it considers many. Top-k always considers exactly k tokens regardless of how the probability is distributed — which can be too few (missing valid options) or too many (including garbage).

---

## KV-Cache — Critical for Performance

**Problem**: At generation step N, the naive approach recomputes attention for ALL N tokens. That's $O(N^2)$ total work — quadratic!

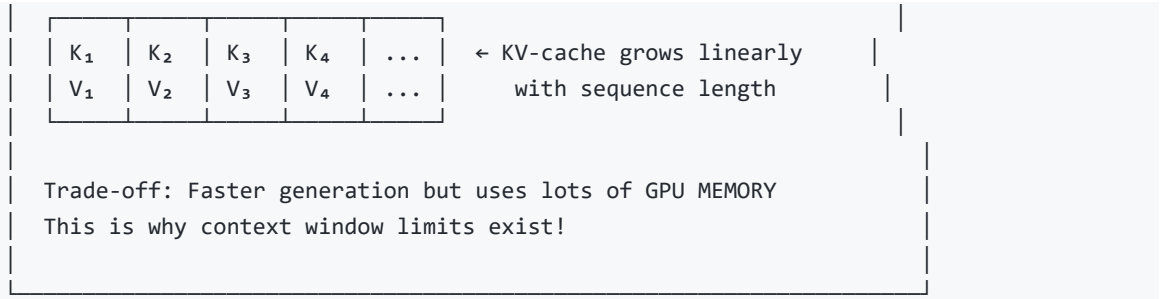
```
Naive approach (no cache):

  Step 1: Compute Q, K, V for ["The"]                    → 1 token
  Step 2: Compute Q, K, V for ["The", "capital"]          → 2 tokens
  Step 3: Compute Q, K, V for ["The", "capital", "of"]    → 3 tokens
  Step 4: Compute Q, K, V for all 4 tokens                → 4 tokens
  ...
  Step N: Compute Q, K, V for all N tokens                → N tokens

  Total work: 1 + 2 + 3 + ... + N = N(N+1)/2 = O(N²)  ← BAD!
```

**Solution**: Cache the K and V vectors from previous steps. Only compute Q for the NEW token.

```
┌──────────────────────────────────────────────────────────────┐
│                        KV-CACHE                              │
│                                                              │
│  Step 1: Compute K₁, V₁ for "The"        → cache [K₁, V₁]        │
│  Step 2: Compute K₂, V₂ for "capital"    → cache [K₁K₂, V₁V₂]    │
│  Step 3: Compute K₃, V₃ for "of"         → cache [K₁K₂K₃,V₁V₂V₃] │
│  Step 4: Compute ONLY Q₄ for "France"                        │
│          Attend Q₄ against cached [K₁K₂K₃K₄, V₁V₂V₃V₄]       │
│                                                              │
│  At each step: O(1) new computation (just the new token)     │
│  Total work:   O(N) instead of O(N²)  ← MUCH better!         │
└──────────────────────────────────────────────────────────────┘
```

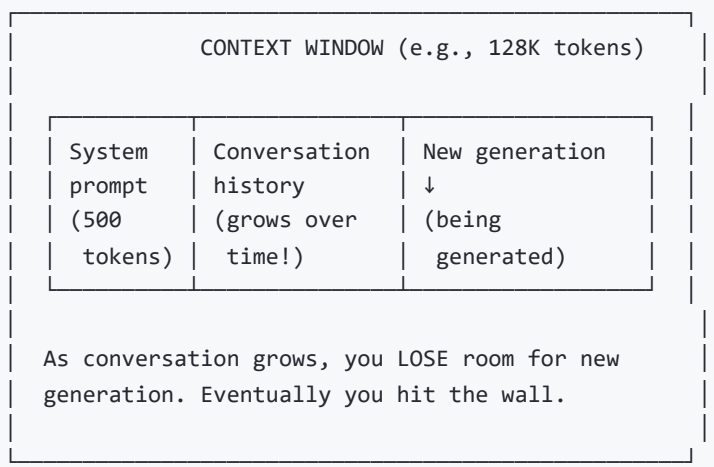



**Memory cost**: For a 70B+ parameter model with 128K context, the KV-cache alone can use **tens of gigabytes** of GPU memory. This is the primary bottleneck for serving LLMs. Frontier models like GPT-5.2 and Claude Opus 4.6 use techniques like GQA (Grouped-Query Attention), sliding window attention, and quantized KV-cache to manage this.

---

### Context Window and Agents

The **context window** is the maximum number of tokens the model can "see" at once — prompt + generated output combined.




**Why this matters for agents**:

- Agent conversations can be LONG (tool calls, observations, reasoning steps)
- Each tool call + result eats into the context window
- Need strategies to manage context: summarization, sliding window, RAG (preview of Day 5)
- This is why context window management is a **core agent engineering skill**

---

## Practice Exercises

## Exercise 1: Tokenization Explorer

```
"""
Exercise 1: Tokenization Explorer
Explore how different tokenizers handle various inputs.
Find surprising edge cases in tokenization.

Requirements: pip install tiktoken
No API key needed.
"""

import tiktoken


def explore_tokenization():
    """Compare tokenization across different model encodings."""
    # Load tokenizers for different models
    enc_gpt4 = tiktoken.encoding_for_model("gpt-4o")
    enc_gpt35 = tiktoken.encoding_for_model("gpt-4")

    print("=" * 60)
    print("TOKENIZATION EXPLORER")
    print("=" * 60)

    # --- Test 1: Basic text ---
    test_texts = [
        "Hello, world!",
        "The capital of France is Paris.",
        "def fibonacci(n): return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)",
        "    " * 10,  # lots of whitespace
        "🎉🎊🎈🎁",  # emojis
        "こんにちは世界",  # Japanese
        "Привет мир",  # Russian
        "2 + 2 = 4",
        "HTTP/1.1 200 OK",
        "aaaaaaaaaa",  # repeated characters
        " a  a   a    a",  # varying whitespace
        "don't won't can't",  # contractions
    ]

    print("\n--- Token counts (GPT-4o / o200k encoding) ---")
    print(f"{'Text':<50} {'Tokens':<8} {'Chars/Token':<12}")
    print("-" * 70)

    for text in test_texts:
        tokens = enc_gpt4.encode(text)
        ratio = len(text) / len(tokens) if tokens else 0
        display = text[:47] + "..." if len(text) > 50 else text
        print(f"{display:<50} {len(tokens):<8} {ratio:<12.2f}")

    # --- Test 2: See actual tokens ---
    print("\n--- Token breakdown for selected texts ---")
    interesting = [
        "Hello, world!",
```

```
        "    " * 4,
        "don't won't can't",
        "def fibonacci(n):",
    ]

    for text in interesting:
        tokens = enc_gpt4.encode(text)
        decoded = [enc_gpt4.decode([t]) for t in tokens]
        print(f"\nText: {repr(text)}")
        print(f"  Tokens ({len(tokens)}): {decoded}")
        print(f"  Token IDs: {tokens}")

    # --- Test 3: Find edge cases ---
    print("\n--- Surprising edge cases ---")
    edge_cases = {
        "Numbers": ("42", "420", "4200", "42000", "4200000"),
        "Spacing": (" hello", "  hello", "   hello", "    hello"),
        "Similar words": ("running", "run", "ran", "runner", "runs"),
        "Programming": ("print()", 'print("hello")', "System.out.println()"),
    }

    for category, texts in edge_cases.items():
        print(f"\n{category}:")
        for text in texts:
            tokens = enc_gpt4.encode(text)
            decoded = [enc_gpt4.decode([t]) for t in tokens]
            print(f"  {text:<30} → {len(tokens)} tokens: {decoded}")

    # --- Test 4: Vocabulary size ---
    print(f"\n--- Vocabulary info ---")
    print(f"GPT-4o / GPT-5 vocabulary size: {enc_gpt4.n_vocab:,} tokens")

    # --- Test 5: Cost estimation ---
    print("\n--- Cost estimation ---")
    sample_texts = {
        "A tweet (280 chars)": "x" * 280,
        "A short email (500 words)": " ".join(["word"] * 500),
        "A page of code (50 lines)": "\n".join(
            ["    result = process(item)  # do something"] * 50
        ),
        "A novel chapter (~5000 words)": " ".join(["word"] * 5000),
    }

    for desc, text in sample_texts.items():
        tokens = enc_gpt4.encode(text)
        cost_in = len(tokens) * 0.000001  # rough GPT-5.1 input pricing ($1/M tokens
        print(f"  {desc:<35} → {len(tokens):>6} tokens (~${cost_in:.4f} GPT-5.1 inpu


if __name__ == "__main__":
    explore_tokenization()
```

## Exercise 2: Sampling Strategy Visualizer

```
"""
Exercise 2: Sampling Strategy Visualizer
Visualize how temperature, top-k, and top-p change a probability distribution.
Uses ASCII art — no matplotlib needed, no API key needed.

Requirements: None (pure Python)
"""

import math
import random


def softmax(logits, temperature=1.0):
    """Apply softmax with temperature scaling."""
    if temperature == 0:
        # Greedy: all probability on the max
        result = [0.0] * len(logits)
        result[logits.index(max(logits))] = 1.0
        return result
    scaled = [x / temperature for x in logits]
    max_val = max(scaled)
    exps = [math.exp(x - max_val) for x in scaled]
    total = sum(exps)
    return [e / total for e in exps]


def apply_top_k(probs, k):
    """Zero out everything except the top-k probabilities, then renormalize."""
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    result = [0.0] * len(probs)
    total = 0.0
    for i, (idx, p) in enumerate(indexed):
        if i < k:
            result[idx] = p
            total += p
    if total > 0:
        result = [p / total for p in result]
    return result


def apply_top_p(probs, p_threshold):
    """Keep smallest set of tokens with cumulative probability >= p."""
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    result = [0.0] * len(probs)
    cumulative = 0.0
    for idx, p in indexed:
        result[idx] = p
        cumulative += p
        if cumulative >= p_threshold:
            break
    total = sum(result)
    if total > 0:
```

```
        result = [p / total for p in result]
    return result


def ascii_bar(prob, max_width=30):
    """Create an ASCII bar for a probability."""
    filled = int(prob * max_width)
    return "\u2588" * filled + "\u2591" * (max_width - filled)


def display_distribution(probs, labels, title=""):
    """Display a probability distribution as ASCII bars."""
    print(f"\n  {title}")
    print(f"  {'Token':<12} {'Prob':>6}  {'Distribution':<30}")
    print(f"  {'-'*12} {'-'*6}  {'-'*30}")
    for label, prob in zip(labels, probs):
        if prob > 0.001:
            bar = ascii_bar(prob)
            print(f"  {label:<12} {prob:>5.1%}  {bar}")
    tokens_shown = sum(1 for p in probs if p > 0.001)
    tokens_hidden = sum(1 for p in probs if 0 < p <= 0.001)
    if tokens_hidden > 0:
        print(f"  ... and {tokens_hidden} tokens with < 0.1% each")


def main():
    print("=" * 65)
    print("  SAMPLING STRATEGY VISUALIZER")
    print("=" * 65)

    # Simulate logits for next token after "The weather today is"
    tokens = [
        "sunny", "nice", "warm", "cold", "great",
        "terrible", "rainy", "hot", "perfect", "awful",
        "beautiful", "cloudy", "windy", "mild", "freezing",
    ]
    # Raw logits (before softmax) — "sunny" is the most likely
    logits = [
        4.5, 3.2, 2.8, 2.1, 1.9,
        1.5, 1.3, 1.1, 0.9, 0.5,
        0.3, 0.1, -0.2, -0.5, -1.0,
    ]

    # --- Demo 1: Temperature ---
    print("\n" + "\u2500" * 65)
    print("  TEMPERATURE EFFECT")
    print("  Context: 'The weather today is ___'")
    print("\u2500" * 65)

    for temp in [0.1, 0.5, 1.0, 2.0]:
        probs = softmax(logits, temperature=temp)
        label = {
            0.1: "T=0.1 (nearly deterministic)",
            0.5: "T=0.5 (focused)",
            1.0: "T=1.0 (normal)",
            2.0: "T=2.0 (creative/random)",
```

```
        }[temp]
        display_distribution(probs, tokens, label)

    # --- Demo 2: Top-k ---
    print("\n" + "—" * 65)
    print("  TOP-K SAMPLING (at T=1.0)")
    print("—" * 65)

    base_probs = softmax(logits, temperature=1.0)
    for k in [1, 3, 5, 10]:
        filtered = apply_top_k(base_probs, k)
        active = sum(1 for p in filtered if p > 0)
        display_distribution(
            filtered, tokens, f"Top-k (k={k}) — {active} tokens active"
        )

    # --- Demo 3: Top-p ---
    print("\n" + "—" * 65)
    print("  TOP-P (NUCLEUS) SAMPLING (at T=1.0)")
    print("—" * 65)

    for p in [0.5, 0.7, 0.9, 0.95]:
        filtered = apply_top_p(base_probs, p)
        active = sum(1 for pr in filtered if pr > 0)
        display_distribution(
            filtered, tokens, f"Top-p (p={p}) — {active} tokens in nucleus"
        )

    # --- Demo 4: Simulate generation ---
    print("\n" + "—" * 65)
    print("  SIMULATED GENERATION (10 samples each)")
    print("—" * 65)

    configs = [
        ("Greedy (T=0)",       {"temperature": 0.0}),
        ("Low temp (T=0.3)",   {"temperature": 0.3}),
        ("Normal (T=1.0)",     {"temperature": 1.0}),
        ("Creative (T=1.5)",   {"temperature": 1.5}),
        ("Top-k=3, T=1.0",    {"temperature": 1.0, "top_k": 3}),
        ("Top-p=0.9, T=1.0",  {"temperature": 1.0, "top_p": 0.9}),
    ]

    for name, config in configs:
        samples = []
        for _ in range(10):
            temp = config.get("temperature", 1.0)
            probs = softmax(logits, temperature=temp) if temp > 0 else softmax(logit
            if "top_k" in config:
                probs = apply_top_k(probs, config["top_k"])
            if "top_p" in config:
                probs = apply_top_p(probs, config["top_p"])

            # Sample from distribution
            r = random.random()
            cumulative = 0.0
            chosen = tokens[-1]
```

```
            for token, prob in zip(tokens, probs):
                cumulative += prob
                if r <= cumulative:
                    chosen = token
                    break
            samples.append(chosen)

        unique = len(set(samples))
        print(f"\n  {name}:")
        print(f"    Samples: {', '.join(samples)}")
        print(f"    Unique tokens: {unique}/10")

    print("\n" + "=" * 65)
    print("  KEY TAKEAWAYS:")
    print("  • Temperature controls the 'sharpness' of the distribution")
    print("  • Top-k fixes the number of candidates (rigid)")
    print("  • Top-p adapts the candidates to the distribution (flexible)")
    print("  • For factual tasks: low temperature (0.0-0.3)")
    print("  • For creative tasks: higher temperature + top-p (0.7-1.0)")
    print("=" * 65)


if __name__ == "__main__":
    main()
```

---

## Exercise 3: Token Probability Analyzer (Optional — Needs API Key)

```
"""
Exercise 3: Token Probability Analyzer
Use OpenAI API to see where the model is confident vs uncertain.

Requirements: pip install openai
Needs: OPENAI_API_KEY environment variable
"""

import os

try:
    from openai import OpenAI
except ImportError:
    print("Run: pip install openai")
    exit(1)


def analyze_logprobs(prompt: str, model: str = "gpt-5.1-codex-mini"):
    """Generate text and analyze per-token probabilities."""
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        print("Set OPENAI_API_KEY environment variable first.")
        print("Export: export OPENAI_API_KEY='sk-...'")
        return
```

```
    client = OpenAI(api_key=api_key)

    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=100,
        logprobs=True,
        top_logprobs=5,
        temperature=1.0,
    )

    choice = response.choices[0]
    content = choice.message.content
    logprobs_data = choice.logprobs.content

    print(f"\nPrompt: {prompt}")
    print(f"Response: {content}")
    print(f"\n{'Token':<20} {'Prob':>8} {'Confidence':<15} {'Alternatives'}")
    print("-" * 80)

    import math

    for token_info in logprobs_data:
        token = token_info.token
        logprob = token_info.logprob
        prob = math.exp(logprob)

        # Confidence indicator
        if prob > 0.9:
            confidence = "████████ SURE"
        elif prob > 0.7:
            confidence = "██████▒▒ high"
        elif prob > 0.4:
            confidence = "████▒▒▒▒ med"
        elif prob > 0.1:
            confidence = "██▒▒▒▒▒▒ low"
        else:
            confidence = "▒▒▒▒▒▒▒▒ guess"

        # Top alternatives
        alts = []
        for alt in token_info.top_logprobs[:3]:
            alt_prob = math.exp(alt.logprob)
            if alt.token != token:
                alts.append(f"{repr(alt.token)}:{alt_prob:.1%}")

        alt_str = ", ".join(alts[:3]) if alts else "(no alternatives)"
        print(f"{repr(token):<20} {prob:>7.1%} {confidence:<15} {alt_str}")


def main():
    print("=" * 60)
    print("TOKEN PROBABILITY ANALYZER")
    print("=" * 60)
```

```
    prompts = [
        "The capital of France is",
        "Write a haiku about programming",
        "What is 17 * 23?",
        "The meaning of life is",
    ]

    for prompt in prompts:
        print("\n" + "=" * 60)
        analyze_logprobs(prompt)


if __name__ == "__main__":
    main()
```

---

## ✓ Day 1 Deliverable Checklist

```
- [ ] Explored tokenization with tiktoken (found 3+ surprising edge cases)
- [ ] Can draw the full Transformer architecture from memory
- [ ] Built a sampling strategy visualizer
- [ ] Can explain: What is a token and why isn't it always a word?
- [ ] Can explain: What are Q, K, V and why do we need all three?
- [ ] Can explain: What is the difference between pre-training, SFT, and RLHF?
- [ ] Can explain: How does temperature affect generation?
- [ ] Can explain: Why does the KV-cache exist?
- [ ] Can explain: What is the feed-forward network's role vs attention's role?
- [ ] Can explain: Why do modern LLMs use decoder-only architecture?
```

▶ 📝 Answer Key (click to expand)

---

Next → Day 2: Advanced Prompt Engineering

Deep dive → Appendix: Advanced Techniques